Password Protection

 (https://github.com/learn-co-curriculum/python-p4-passwords)  (https://github.com/learn-co-curriculum/python-p4-passwords/issues/new)

Learning Goals

- Explain why it's a bad idea to store passwords in plaintext.
- Write code to store and verify hashed, salted passwords.
- Use SQLAlchemy and Bcrypt to store and authenticate user login credentials securely.

Key Vocab

- **Identity and Access Management (IAM)**: a subfield of software engineering that focuses on users, their attributes, their login information, and the resources that they are allowed to access.
- **Authentication**: proving one's identity to an application in order to access protected information; logging in.
- **Authorization**: allowing or disallowing access to resources based on a user's attributes.
- **Session**: the time between a user logging in and logging out of a web application.
- **Cookie**: data from a web application that is stored by the browser. The application can retrieve this data during subsequent sessions.

Introduction

It's quite difficult to manage passwords securely. About once a month, there is another big hack in the news, and all the passwords and credit cards from some poor site show up on the dark web.

Flask provides us with tools to store passwords securely so that even if our database is compromised, no one can gain access to users' actual passwords.

The Problem with Passwords

Let's imagine a `post()` method that does very simple authentication. It goes like this:

```
class Login(Resource):

    def post(self):

        username = request.get_json()['username']
        user = User.query.filter(User.username == username)

        password = request.get_json()['password']
        if password == user.password:
            session['user_id'] = user.id
            return user.to_dict(), 200

        return {'error': 'Invalid username or password'}, 401
```

We find the user in the database by their username, check to see if the provided password is equal to the password stored in the database, and, if it is, set `user_id` in the `session`.

This is tremendously insecure because you then have to store all your users' passwords in the database, unencrypted.

Never do this.

Even if you don't care about the security of your site, people have a strong tendency to reuse passwords. That means that the inevitable security breach of your site will leak passwords which some users also use for Gmail. Your `users` table probably has an `email` column. This means that, if I'm a hacker, getting access to your database has given me the Internet equivalent of the house keys and home address for some (probably surprisingly large) percentage of your users.

Hashing Passwords

So how do we store passwords if we can't store passwords?

Instead of storing users' passwords in plain text, we store a hashed version of them. A *hash* is a *fixed-length* output computed by feeding a string to a *hash function*. Hash functions have the property that they will always produce the same output given the same input.

A helpful analogy for a hash function is making a smoothie. If I put the exact same ingredients into the blender, I'll get the exact same smoothie every time. But there's no way to reverse the operation, and get back the original ingredients from the smoothie.

Hash functions work in a similar way: given the same input, they'll always produce the same output; and there's no way to reverse the output and recreate the original input.

You could even write a hash function yourself. Here's a very simple one:

```
def simple_hash(input):
    return sum(bytearray(input, encoding='utf-8'))
```

This `simple_hash()` function just finds the sum of the bytes that comprise the string. It satisfies the criterion that the same string always produces the same result. (It doesn't quite meet the "fixed-length output" requirement for hashes, but for demo purposes, it'll do.)

We could imagine using this function to avoid storing passwords in the database. Our `User` model and `SessionsController` might look like this:

```
# server/models.py
from sqlalchemy.ext.hybrid import hybrid_property

class User(db.Model, SerializerMixin):
    __tablename__ = 'users'

    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String, unique=True)
    _password_hash = db.Column(db.String, nullable=False)
```

```
    articles = db.relationship('Article', backref='user')

    def __repr__(self):
        return f'User {self.username}, ID {self.id}'

    # this is a special property decorator for sqlalchemy
    # it leaves all of the sqlalchemy characteristics of the column in place
    @hybrid_property
    def password_hash(self):
        return self._password_hash

    # setter method for the password property
    @password.setter
    def password_hash(self, password):
        self._password_hash = self.simple_hash(password)

    # authentication method using user and password
    def authenticate(self, password):
        return self.simple_hash(password) == self.password_hash

    # simple_hash requires no access to the class or instance
    # let's leave it static
    @staticmethod
    def simple_hash(input):
        return sum(bytearray(input, encoding='utf-8'))


# server/app.py
class Login(Resource):

    def post(self):
```

```
    username = request.get_json()['username']
    user = User.query.filter(User.username == username)

    password = request.get_json()['password']

    if user.authenticate(password):
        session['user_id'] = user.id
        return user.to_dict(), 200

    return {'error': 'Invalid username or password'}, 401
```

In this world, we have saved the password hashes in the database. We are not storing the passwords themselves.

With the code above, a user's password is set by calling `user.password = *new_password*` . Presumably, this would happen programmatically, but this could be accomplished manually by an administrator as well.

`simple_hash()` is, as its name suggests, a pretty simple hash function to use for this purpose. It's a poor choice because similar strings hash to similar values. If my password was `Joshua` , you could log in as me by entering the password `Jnshub` . Since 'n' is one less than 'o' and 'b' is one more than 'a', the output of `simple_hash()` would be the same. This is known as a *collision*. With `simple_hash()` as our hashing function, there would be many, similar, variants of our `Joshua` password (many collisions) that could be used successfully to access the account, making our authentication process much less secure.

Unfortunately, collisions are inevitable when you're writing a hash function, since hash functions usually produce either a 32-bit or 64-bit number, and the space of all possible strings is much larger than either `2**32` or `2**64` . Fortunately, however, smart people who have thought about this a lot have written a lot of different hash functions that are well-suited to different purposes. And nearly all hash functions are designed with the quality that strings that are similar, but not the same, will hash to significantly different values.

Instead, Flask uses a library called Bcrypt. Bcrypt is designed with these properties in mind:

1. Bcrypt hashes similar strings to very different values.
2. It is a *cryptographic hash*. That means that, if you have an output in mind, finding a string which produces that output is designed to be "very difficult." "Very difficult" means "even if Google put all their computers on it, they couldn't do it."

3. Bcrypt is designed to be slow. It is intentionally computationally expensive.

The last two features make Bcrypt a particularly good choice for passwords. (2) means that, even if an attacker gets your database of hashed passwords, it is not easy for them to turn a hash back into its original string. (3) means that, even if an attacker has a dictionary of common passwords to check against, it will still take them a considerable amount of time to check for your password against that list.

The **Flask-Bcrypt extension** (https://flask-bcrypt.readthedocs.io/en/1.0.1/) is open source, and their documentation has some excellent examples that demonstrate this functionality. If you're interested in exploring more, their docs and source code are a great resource.

Salt

But what if our attackers have done their homework?

Say I'm a hacker. I know I'm going to break into a bunch of sites and get their password databases. I want to make that worth my while.

Before I do all this breaking and entering, I'm going to find the ten million most common passwords and hash them with Bcrypt. I can do around 1,000 hashes per second, so that's about three hours. Maybe I'll do the top five hundred million just to be sure.

It doesn't really matter that this is going to take long time to run — I'm only doing it once. Let's call this mapping of strings to hash outputs a **"rainbow table"** (https://en.wikipedia.org/wiki/Rainbow_table).

Now, when I get your database, I just look and see if any of the passwords in it are in my rainbow table. If they are, then I know the password.

Going back to our smoothie analogy, this would be the equivalent of someone taking all the possible combinations of smoothie ingredients and running them through the blender to create a giant collection of smoothies. By tasting all the smoothies, they could figure out which original ingredients were used to make the smoothie they're trying to identify.

The solution to the rainbow table problem is *salting* our passwords. A salt is a random string prepended to the password before hashing it. It's stored in plain text next to the password, so it's not a secret. But the fact that it's there makes an attacker's life much more difficult: it's very unlikely that I constructed my rainbow table with your particular salt in mind, so I'm back to running the hash algorithm over and over as I guess passwords. And, remember, Bcrypt is designed to be expensive to run.

Let's update our app to configure Bcrypt and the `User` model to use it:

```python
# server/app.py
from flask.ext.bcrypt import Bcrypt
# instantiate Bcrypt with app instance
bcrypt = Bcrypt(app)

class Login(Resource):

    ...
```

```python
# server/models.py
from sqlalchemy.ext.hybrid import hybrid_property
from app import bcrypt

class User(db.Model, SerializerMixin):
    __tablename__ = 'users'

    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String, unique=True)
    _password_hash = db.Column(db.String, nullable=False)

    articles = db.relationship('Article', backref='user')

    def __repr__(self):
        return f'User {self.username}, ID {self.id}'

    @hybrid_property
    def password_hash(self):
        return self._password_hash

    @password_hash.setter
    def password_hash(self, password):
```

```
        # utf-8 encoding and decoding is required in python 3
        password_hash = bcrypt.generate_password_hash(
            password.encode('utf-8'))
        self._password_hash = password_hash.decode('utf-8')

    def authenticate(self, password):
        return bcrypt.check_password_hash(
            self._password_hash, password.encode('utf-8'))
```

Our `users.password_hash` column really stores two values: the salt and the actual return value of Bcrypt. We just concatenate them together in the column and use our knowledge of the length of salts — Bcrypt always produces 29-character strings — to separate them.

After we've loaded the User, we find the salt which we previously stored in their `password_hash` column. We run the password we were given in `get_json()` through Bcrypt along with the salt we read from the database. If the results match, you're in. If they don't, no dice.

Conclusion

When dealing with users' passwords, it's important for security that we never store passwords in our database directly in plain text. Instead, we can use a trusted library like Bcrypt to help keep our users' passwords safe.

Check For Understanding

Before you move on, make sure you can answer the following questions:

▶ *1. What setup steps do you need to complete to use Bcrypt in your Flask app?*

▶ *2. What two things does Bcrypt do to secure passwords?*

Resources

- What are cryptographic hash functions? - Synopsys (https://www.synopsys.com/blogs/software-security/cryptographic-hash-functions/)
- Wikipedia — Rainbow Table (https://en.wikipedia.org/wiki/Rainbow_table)
- Bcrypt USENIX paper (https://www.usenix.org/legacy/event/usenix99/provos/provos.pdf)
- Flask-Bcrypt (https://flask-bcrypt.readthedocs.io/en/1.0.1/)